

开源协作 · 工程效能 · 互动手册

工程习惯规范 自学与落地手册

从“能跑的原型”到“能联调、能交付的工程”——开源仓库治理与工程最佳实践

这本手册回答两个问题：我们现在哪里做得还不够好？怎么做能更好？

体检快照基于三个仓库的实测数据 · v2 互动版 · 2026-06 · 左侧目录可跳转， ☒ 清单可勾选并自动保存，代码块可一键复制

0 这份文档解决什么问题

我们几个方向已经各自跑出了原型。但“原型能跑”和“几个仓库能拼成一个系统”是两回事。接下来各条线要并行往前推，很快就要做 `host` ↔ `协调器` ↔ `Council` 的端到端联调。如果现在不把**工程习惯**和**跨仓契约**立起来，等到联调时一定会在“格式冲突 / 接口对不上 / 谁也不敢改 main / 没有埋点数据”里翻车。

这份手册做三件事：

1. **照镜子**——用当天实测数据，给三个仓库做一次诚实体检（第 1 章）。
2. **给标准**——一个健康开源仓库“长什么样”，以及必须引入的脚手架配置项，全部可复制粘贴（第 2–4 章）。
3. **能落地**——把协作流、契约管理、项目管理讲清楚，并给出“本周每个仓库要补的事”清单（第 5–8 章）。

怎么读：赶时间的同学直接跳到 第 4 章“**必备脚手架清单**”抄配置，再看 第 9 章“**本周落地清单**”照着做即可。带 ★ 的是本周必须完成项。

目录

1. 现状体检：我们三个仓库的 15 个症结
2. 开源视角：一个健康仓库长什么样
3. 为什么这些配置不是“洁癖”，而是基本要求
4. 必备脚手架清单（可复制粘贴）★
5. 协作流：基于 PR 的分支工作流（把 6.2/6.3/6.4 落到 GitHub）★
6. 跨仓契约管理：让“契约冻结”真正生效★
7. 项目管理：Issue / 标签 / 看板 / 里程碑
8. 仓库治理：搬进一个 GitHub Org
9. 本周落地清单：每个仓库要补的 8 件事★
10. 附录：一键脚手架 & 推荐阅读对照表

1 现状体检：我们三个仓库的 15 个症结

数据来源：2026-06-16 对 `acp-client-prototype`、`hci-ide-mvp`、`newide-scaffold` 三仓的本地 git 历史 + GitHub 元数据实测。

#	症结	实测证据	不改的后果
1	三仓三主，没有统一 Org	三个仓库分别挂在三位同学的个人 GitHub 账号下 (DWangSE / ExtraZhangYC / Neighhhbor)	权限与可见性碎片化；bus factor = 1，谁失联谁的仓就锁死
2	高危 全员直推 main，0 个 PR	三仓 Pull Request 总数 = 0，所有提交直接落在 <code>main</code>	无评审、无门禁——这正是代码复审要避免的
3	main 分支零保护	可直推 / 可强推；hci 有一条 <code>feature-update-zyc</code> 分支领先 main 1 个提交， 从未合并	误覆盖历史；分支漂移，工作“孤儿化”
4	高危 仓库里塞二进制垃圾	hci 提交了 <code>gh.zip</code> (5.8 MB)、 <code>*.tsbuildinfo</code> ；acp 提交了 <code>my-session.json</code> (1.1 MB 会话转储)	克隆体积膨胀、泄漏本地状态、污染每次 diff
5	.gitignore 不达标	hci 的 <code>.gitignore</code> 仅 5 行，漏掉 <code>tsbuildinfo</code> 等构建产物	构建缓存反复入库，diff 噪声大
6	Git 身份没配	hci 全部提交作者显示为 <code>unknown <...@qq.com></code>	贡献不可追溯，review 时找不到人
7	提交信息随意	充斥 <code>Doc update</code> / <code>Prototype</code> / <code>License</code> / <code>完成本地功能小幅修改</code>	历史不可读，无法回溯、无法自动生成 changelog
8	高危 完全没有 CI	三仓均无 <code>.github/workflows/</code>	没有自动 lint / 类型 / 测试 / 构建门禁，坏代码直达 main， 联调时必爆
9	工具链各搞各的	newide 用 <code>pnpm + eslint + prettier</code> ；acp 用 <code>npm</code> 仅 build；hci 用 <code>npm</code> 仅 dev/build	三仓三套规则，联调时陷入格式 / 类型地狱
10	缺测试脚本	acp 有 <code>tests/</code> 目录却无 <code>test</code> 脚本 / 测试框架；hci 无 test	无法自动验证，回归全靠手测
11	治理文件全缺	三仓均无 CONTRIBUTING / PR 模板 / Issue 模板 / CODEOWNERS / .editorconfig / .nvrc；2/3 无 LICENSE	新人无从下手，协作无章可循
12	Issue 质量参差	newide 范本 (任务/目标/完成标准 + area 标签)；hci 有 完全重复 的 issue (#4/#5)、无标签、标题含糊 (“UI 美化” “技术栈选择?”)；acp 作为一条工作流 0 issue	任务不可见、不可追踪、易重复劳动
13	标签体系不统一	newide 自建 <code>area: b/c/d</code> (还用了 全角冒号 “:”)、 <code>integration</code> ；hci 仍是 GitHub 默认标签	跨仓无法统一筛选、无法做统一看板
14	高危 契约没有“单一可信源”	契约散落：acp 删掉了 <code>SPEC.md</code> 、newide 把契约写进 <code>src/*/contract.ts</code> 、hci 写在 <code>plan.md</code> 。没有一处集中冻结的契约文档	“11 项跨方向契约”无法真正冻结，各线各写各的接口
15	高危 必埋点清单缺失	三仓未发现任何 埋点 / telemetry / analytics 工件；WS4 的“必埋点清单”产物不存在	埋点接入无依据，三期评测拿不到数据

一句话诊断：我们三个“能演示的原型”，但还没有“一套能协同的工程”。上面 15 条里，**有 11 条本周就能用半天到一天补齐**（脚手架 + 配置），剩下 4 条是流程习惯，从本周开始执行就能养成。

已经做对的地方（值得全队对齐）

- **newide-scaffold** 是三仓里工程化最好的：用了 pnpm + ESLint + Prettier，把契约写成了 `contract.ts`，Issue 拆分按 `area B/C/D` + `integration` 标签，Issue 正文有“任务/目标/完成标准”。**本手册的很多“正面范例”直接取自它，建议另外两仓对齐它的做法。**
- **hci-ide-mvp** 已经在用 Issue 记录验证结论，方向对，只是需要去重 + 加标签 + 规范正文。
- **acp-client-prototype** 有清晰的 `src/driver`、`src/adapters/adapters`、`src/hook-gate` 分层和 API 文档，缺的是把 SPEC 找回来并冻结。

2 开源视角：一个健康仓库长什么样

判断一个仓库是否“工程上健康”，开源社区有一套约定俗成的“及格线”。对照一下，缺什么一目了然：

健康仓库根目录（★ = 我们目前普遍缺失）

```
my-repo/
├─ .github/
│  └─ workflows/ci.yml          ★ CI: 每个 PR 自动 lint+类型+测试+构建
│  └─ PULL_REQUEST_TEMPLATE.md ★ PR 自检清单
│  └─ ISSUE_TEMPLATE/          ★ Issue 模板（任务/缺陷）
│  └─ CODEOWNERS                ★ 谁负责哪块代码，自动指派 reviewer
├─ src/                         源码（按领域分层）
├─ tests/                       测试（要有能跑的 test 脚本）
├─ .gitignore                   ★ 完整，挡住产物/缓存/密钥/会话
├─ .editorconfig                ★ 跨编辑器统一缩进/编码/换行
├─ .nvmrc                       ★ 锁定 Node 版本
├─ eslint.config.js             ★ 统一代码规则
├─ prettier.config.js           ★ 统一格式
├─ tsconfig.json                strict 模式
├─ package.json                 统一 scripts + packageManager + engines
├─ README.md                   它是什么 / 怎么跑 / 怎么贡献
├─ CONTRIBUTING.md              ★ 协作规范（分支/提交/PR）
└─ LICENSE                      ★ 开源许可（2/3 仓缺）
```

记忆口诀——任何人 clone 你的仓库后，应该能在 5 分钟内完成三件事：①读 README 知道这是什么、怎么起；②跑一条命令把环境装好（版本被锁定）；③改一行代码、提一个 PR，CI 自动告诉他过没过。我们现在三件都做不到。

3 为什么这些配置不是“洁癖”，而是基本要求

配置项	它在防什么真实事故	对联调/集成的意义
统一 Prettier + ESLint	防止“A 用 Tab、B 用空格、C 不加分号”导致每次合并都是几百行格式 diff	四仓格式一致，review 只看逻辑不看排版
TypeScript strict	防止 <code>undefined is not a function</code> 这类运行时才炸的低级错	接口对接时类型先帮你挡掉一半 bug
CI 门禁	防止“我本地是好的”——坏代码进不了 main	main 永远可联调，集成时不用先花两天修构建
PR + Review	防止单点知识、防止悄悄改坏别人依赖的接口	契约变更必须经过对方 reviewer 同意
.gitignore + 不传二进制	防止 5.8MB 的 <code>gh.zip</code> 、1.1MB 会话转储永远留在 git 历史里删不掉	仓库轻、clone 快、diff 干净
Conventional Commits	防止 <code>Doc update</code> 这种无信息历史	能自动生成 changelog、能按提交追溯谁动了契约
契约单一可信源	防止“我以为字段叫 <code>taskId</code> ，你那边叫 <code>task_id</code> ”	11 项契约真正“冻结”，各线不跑偏

投入产出比：把脚手架立起来，每个仓库一次性投入约 **半天**；它在联调时给你省下的，是 **2-3 天的“为什么拼不起来”排查**。

4 必备脚手架清单（可复制粘贴）★

下面以 **TypeScript + pnpm** 为统一基线（向工程化最好的 newide 对齐）。逐个文件抄进每个仓库即可。

4.1 锁定工具链：.nvmrc + package.json

```
.nvmrc
```

```
20
```

```
package.json（关键字段——统一 scripts / 锁包管理器 / 锁 Node）
```

```
{
  "name": "@ade/wsX-xxx",
  "packageManager": "pnpm@9.12.0",
  "engines": { "node": ">=20", "pnpm": ">=9" },
  "scripts": {
    "dev": "...",
    "build": "tsc -b",
    "typecheck": "tsc --noEmit",
    "lint": "eslint .",
    "format": "prettier --check .",
    "format:fix": "prettier --write .",
    "test": "vitest run",
    "verify": "pnpm lint && pnpm typecheck && pnpm test"
  }
}
```

统一约定：每个仓库都必须能跑 `pnpm verify`（lint + 类型 + 测试一把过）。CI 跑的就是它，本地提交前也跑它。

4.2 .gitignore（完整版，挡住产物/缓存/密钥/会话）

```
.gitignore
```

```
# deps & build
node_modules/
dist/
build/
coverage/
*.tsbuildinfo
.vite/

# env & secrets — 永远不要进仓
.env
.env.*
!.env.example

# 本地状态 / 会话转储 / 归档（正是我们踩过的坑）
*.session.json
my-session.json
*.zip
*.tar.gz
*.log

# editor & OS
.DS_Store
```

```
.idea/  
.vscode/*  
!.vscode/extensions.json
```

已经进了仓的垃圾要“洗掉”，不是删一下就行：`gh.zip`、`my-session.json` 仍在 git 历史里。先 `git rm --cached <file>` 停止追踪并提交；体积大的（`gh.zip`）建议用 `git filter-repo` 从历史彻底移除（见附录）。

4.3 .editorconfig (跨编辑器统一)

```
.editorconfig
```

```
root = true  
[*]  
charset = utf-8  
end_of_line = lf  
insert_final_newline = true  
trim_trailing_whitespace = true  
indent_style = space  
indent_size = 2
```

4.4 ESLint + Prettier (统一规则与格式)

```
prettier.config.js
```

```
export default {  
  semi: true, singleQuote: true, printWidth: 100, trailingComma: 'all',  
};
```

```
eslint.config.js (ESLint 9 flat config, TS 项目通用)
```

```
import js from '@eslint/js';  
import ts from 'typescript-eslint';  
import prettier from 'eslint-config-prettier';  
  
export default ts.config(  
  js.configs.recommended,  
  ...ts.configs.recommended,  
  prettier, // 关闭与 Prettier 冲突的规则  
  { ignores: ['dist/', 'node_modules/', 'coverage/'] },  
);
```

4.5 TypeScript strict

tsconfig.json (关键项)

```
{
  "compilerOptions": {
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "noImplicitOverride": true,
    "module": "NodeNext",
    "target": "ES2022",
    "skipLibCheck": true
  }
}
```

4.6 提交前自动门禁: Husky + lint-staged + commitlint

这一步把“规范”从“自觉”变成“自动”——提交时自动格式化、提交信息不规范直接拦下。

安装

```
pnpm add -D husky lint-staged @commitlint/cli @commitlint/config-conventional
pnpm exec husky init
```

.husky/pre-commit

```
pnpm exec lint-staged
```

.husky/commit-msg

```
pnpm exec commitlint --edit "$1"
```

package.json 追加

```
"lint-staged": {
  "*.ts,tsx,js,json,md": ["prettier --write"],
  "*.ts,tsx": ["eslint --fix"]
}
```

commitlint.config.js

```
export default { extends: ['@commitlint/config-conventional'] };
```

4.7 CI: GitHub Actions (每个 PR 自动验收) ★ 最关键的一项

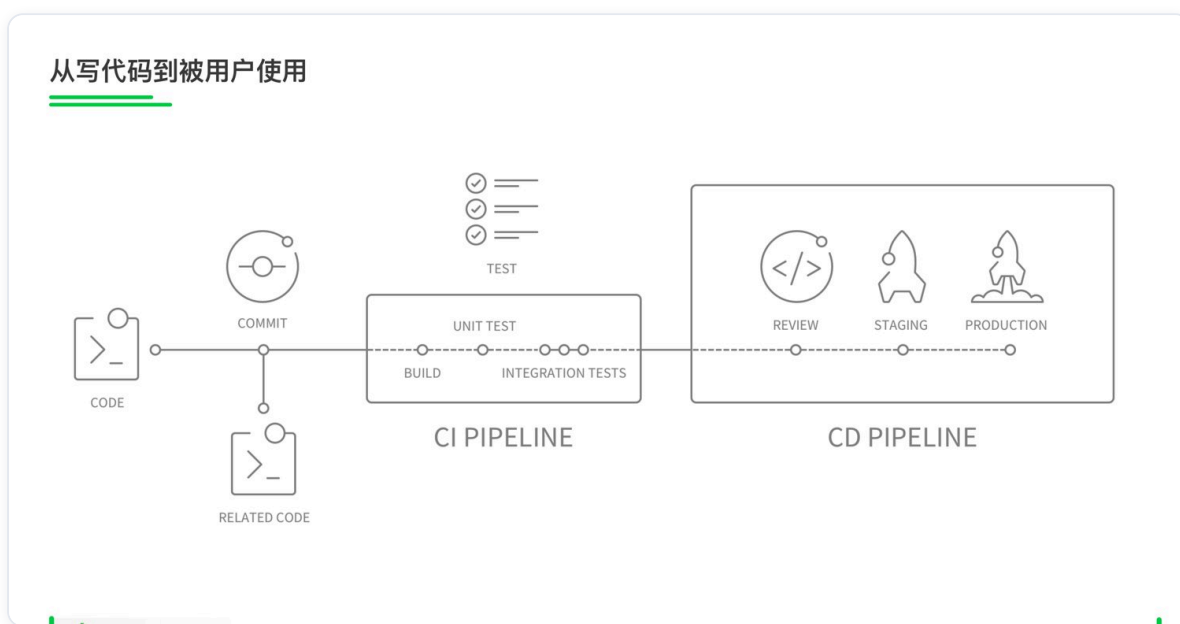
.github/workflows/ci.yml

```
name: CI
on:
  pull_request:
  push:
    branches: [main]
jobs:
  verify:
```



```
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
  - uses: pnpm/action-setup@v4
    with: { version: 9 }
  - uses: actions/setup-node@v4
    with: { node-version: 20, cache: 'pnpm' }
  - run: pnpm install --frozen-lockfile
  - run: pnpm verify          # lint + typecheck + test 一把过
```

配合分支保护（第 8 章）：把这个 **verify** 设为“合并前必须通过的检查”。这样无论谁提 PR，没过 CI 就合不进 main——这是联调不翻车的**单点最大保障**。



代码从写出到上线的全过程：每次提交先过 CI（构建 / 单元测试 / 集成测试），通过后才进入评审与发布——CI 就是这条流水线最前面的自动关卡。

4.8 协作治理文件

.github/PULL_REQUEST_TEMPLATE.md

```
## 做了什么
<!-- 一句话说清这个 PR 的目的 -->

## 关联
Closes #<issue 号> | 涉及契约: <契约编号/无>

## 自检清单
- [ ] 本地 `pnpm verify` 通过
- [ ] 改动是小而聚焦的 (< 400 行优先)
- [ ] 若改了对外接口/契约, 已通知依赖方并更新契约文档
- [ ] 新增/变更逻辑有对应测试或验证说明
```

.github/ISSUE_TEMPLATE/task.md (沿用 newide 的好格式)

```
---
name: 任务
about: 一个可交付的工作项
labels: ''
```

```
---  
**任务:** 一句话描述要做什么  
**目标:** 为什么做、产出什么  
**完成标准 (DoD): **  
- [ ] ...  
- [ ] ...  
**不做:** 明确边界, 避免范围蔓延
```

```
.github/CODEOWNERS
```

```
# 改到这些路径会自动请对应负责人 review  
/src/           @模块负责人           # @ 后面换成真实 GitHub 账号  
/contracts/     @契约负责人A @契约负责人B
```

5 协作流：基于 PR 的分支工作流 ★

在 GitHub 上，“提交一处改动、请别人审查后再合并”这件事叫 **Pull Request (PR)**。本章讲的就是怎么用好 PR，把一次改动安全地合进 main。

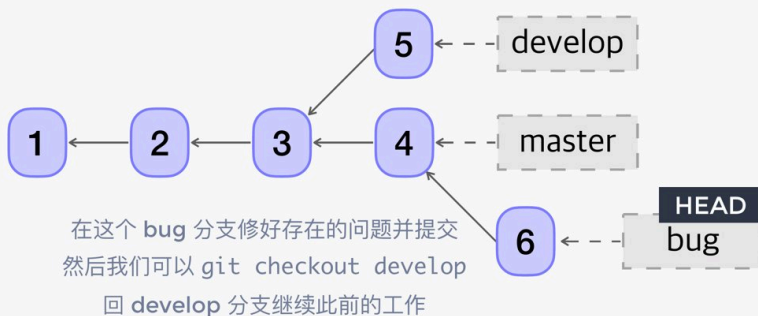
5.1 分支模型：短命特性分支 + 受保护 main

- `main` 永远可用、永远绿（CI 通过）。没有人直接往 main 推。
- 每个任务从 main 切一条**短命分支**，做完发 PR，合并后即删。分支命名：`<类型>/<简述>`，例如 `feat/driver-handshake`、`fix/board-reset`。
- 分支别活太久——拖越久越难合。原则：**一个分支 ≈ 一个 Issue ≈ 1–3 天的活**。

对照我们的现状：hci 那条 `feature-update-zyc` 就是“长命分支 + 没走 PR”的典型——它领先 main 一个提交却合不回，工作有变孤儿的风险。这正是要避免的。

正在写新功能的时候，要插入其他工作怎么办？

未提交的修改



短命特性分支：从主干切出一条分支处理插入进来的工作（例如临时修 bug），完成后再合回主干并删除分支。

5.2 提交规范：Conventional Commits

格式：`<type>(<scope>): <subject>`

`feat(driver)`: 支持 3 个 driver 的握手与心跳

`fix(board)`: Reset 后泳道未清空

`docs(contract)`: 冻结 TaskLifecycle v1

`chore(ci)`: 接入 lint-staged 预提交门禁

type: `feat|fix|docs|refactor|test|chore|build|ci`

好处：历史可读、能自动生成 changelog、能一眼看出哪些提交动了契约（`docs(contract)` / `feat(...)`）。

在开发过程中关注任务计划

每次开发前应该先创建对应到某个 Issue 的开发分支用于完成对应的 Issue

如果开发过程中发生了预估时间的增加，需要及时在 Issue 中更新 `/estimate 2w 3d`

每 commit 一部分工作，要在对应 Issue 写明这部分工作用时 `/spend 2w 3d`

每一次 commit 的时候，需要写明关联到哪个 Issue

```
git commit -m "Refactor: Fix indent style. Closes #12"
```

首次提交用 Closes 关联到 Issue ID
#前空格不能丢

```
git commit -m "Feature: Add support to course switch. Ref #182"
```

已经关闭的 Issue 或非首次提交用 Ref

在提交信息里用 Closes #12 / Ref #182 关联到 Issue (GitHub 同样支持这种写法)，PR 合并后会自动关闭对应 Issue。

5.3 PR 规范 (怎么提)

- **小而聚焦**：一个 PR 只做一件事，优先 < 400 行。大改拆成多个 PR。
- **关联 Issue**：PR 描述写 `Closes #12`，合并后自动关闭 Issue。
- **填模板 + 自检**：本地先 `pnpm verify` 过，再发 PR。
- **动了契约要打招呼**：在 PR 里 @ 依赖方，并更新契约文档。

5.4 Code Review 规范 (怎么看)

提 PR 的人

- 自己先 review 一遍 diff 再请人看
- PR 描述讲清 “为什么这么改”
- 对 reviewer 的每条评论都回应 (改了/解释/记 issue)

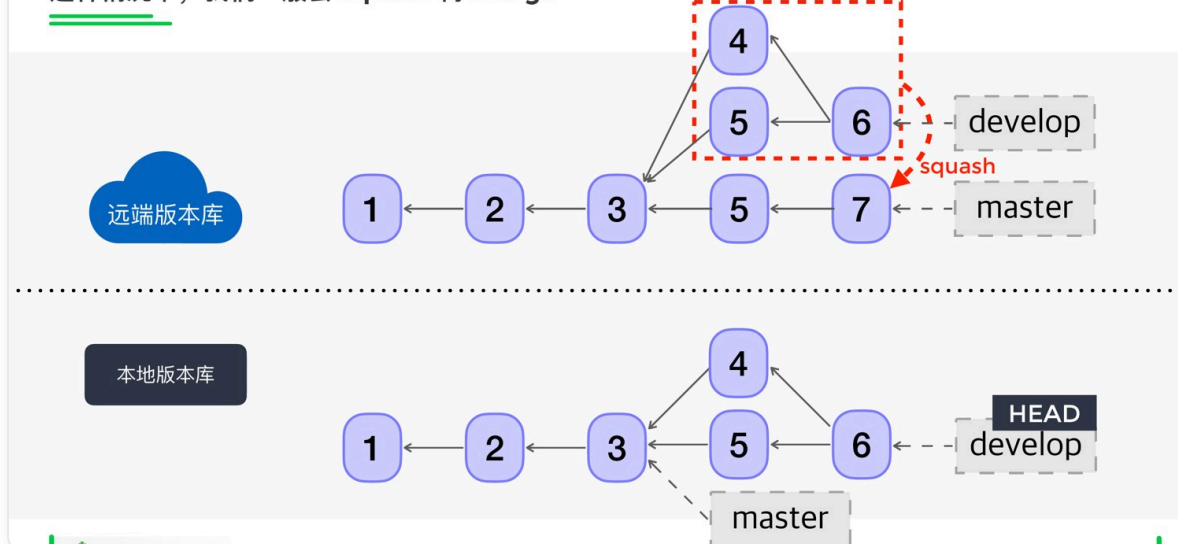
Review 的人

- 24 小时内给反馈，别让 PR 烂掉
- 对事不对人，指出问题给出建议
- 区分 “必须改 (blocking)” 和 “建议 (nit)”
- 至少 1 个 approve 才能合

5.5 合并策略：Squash 为主

- **Squash & merge** 作为默认：一个 PR 在 main 上压成一个干净提交，历史清爽。
- 合并前用 `rebase` 把分支更新到最新 main，减少冲突。
- 合并后删分支，保持分支列表干净。

这种情况下，我们一般会 **squash 再 merge**



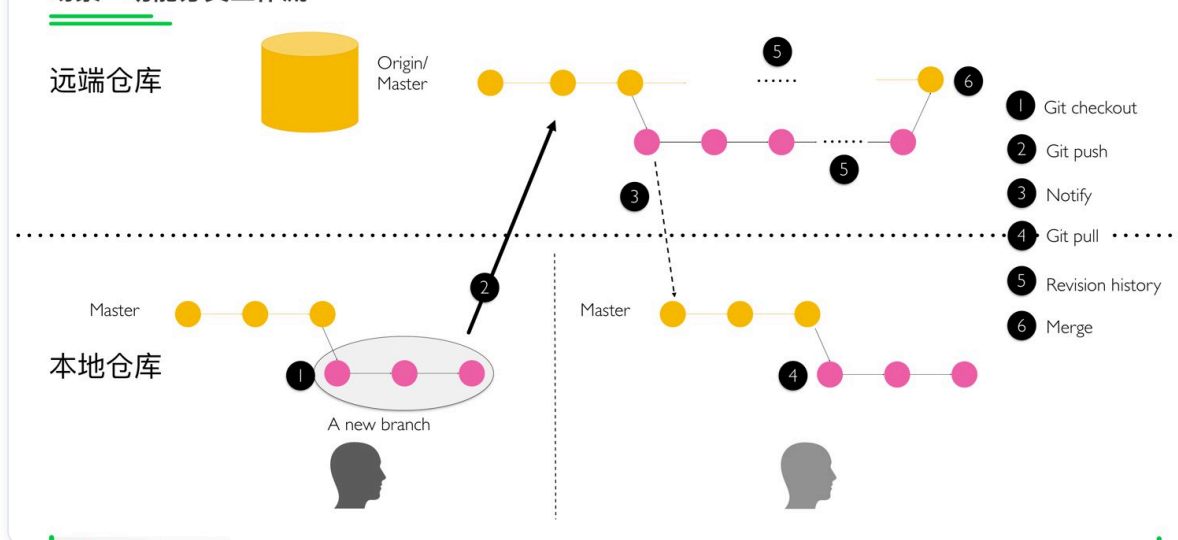
Squash: 把一条分支上零散的多个提交，压成主干上的一个干净提交，主干历史保持清爽。

5.6 一个完整闭环

从认领到合并

1. 在看板认领一个 Issue (assign 给自己, 拖到 In Progress)
2. `git switch -c feat/statemachine-roleloop main`
3. 写代码 + 写测试, 按 Conventional Commits 多次小提交
4. `git push -u origin feat/statemachine-roleloop`
5. 在 GitHub 开 PR, 填模板, Closes #, 请 reviewer
6. CI 自动跑 verify; reviewer review; 按意见改
7. 绿灯 + approve → Squash & merge → 分支自动删除
8. Issue 自动关闭, 看板自动流转到 Done

场景：功能分支 workflow



功能分支协作闭环：本地切分支开发 → push 到远端 → 通知 → 另一人 pull / 评审 → merge 回主干。一次完整的多人协作就是这样跑通的。

6 跨仓契约管理：让“契约冻结”真正生效 ★

契约对齐周的核心产物是“11 项跨方向契约冻结”。但我们现在契约是**散的**：acp 把 SPEC 删了、newide 写在代码里、hci 写在 plan 里。这样无法“冻结”——因为没有一个地方是权威的。

6.1 建立“单一可信源 (Single Source of Truth)”

选一个地方放所有跨方向契约，三种方案，按成本从低到高：

方案	做法	适合
A. 契约文档仓 本周推荐	Org 下建 <code>ade-contracts</code> 仓，放 11 份契约 markdown + 必埋点清单，每次变更走 PR	立刻能做，零代码，今晚就能建
B. 契约目录	各仓内统一 <code>/contracts</code> 目录，但以其中一份为准，定期同步	过渡方案
C. 契约 npm 包	把 <code>contract.ts</code> (类型/schema) 发成 <code>@ade/contracts</code> 包，各仓依赖它	项目中后期，类型级强约束

本周就做 A，并向 C 演进：newide 已经有 `src/*/contract.ts` 了——把它们抽出来当 `@ade/contracts` 的种子，是最省力的路径。

6.2 每份契约长什么样（模板）

contracts/C03-task-lifecycle.md

```
# C03 · Task 生命周期契约
- 状态: 🚫 FROZEN v1 (2026-06-16)    负责人: @xxx
- 生产方: 协调器(WS2)    消费方: host(WS1)、看板(WS3)
## 数据结构
```ts
interface TaskEvent { taskId: string; state: 'pending'|'running'|'blocked'|'done'; ts: number }
...
变更规则
- 冻结后如需改: 开 PR + 标 `contract-change` + 所有消费方 approve 才能合
变更记录
- v1 2026-06-16 冻结
```

### 6.3 必埋点清单也是一种契约（WS4 的依据）

必埋点清单现在**不存在**，这是 WS4 没法接入的根因。它应该和契约放在一起，明确：

contracts/telemetry-required.md

事件名	触发时机	必带字段	产生方
task.created	任务创建	taskId, role, ts	WS2
driver.invoked	调用 driver	taskId, driverId, ts	WS1
gate.decided	Council 出决策	taskId, decision, ts	WS2
board.rendered	看板渲染	taskId, lane, ts	WS3

**冻结的标准：**一份契约“冻结” = 它在契约仓里、状态标 🚫 **FROZEN**、写明生产方/消费方、且所有消费方在 PR 里 approve 过。没满足这几条的，本周还是“草稿”，不是“冻结”。

## 7 项目管理：Issue / 标签 / 看板 / 里程碑

newide 在这块已经做得很好，全队对齐它的做法即可。

### 7.1 Issue：每件事都可见、可追踪

✓ **好范例 (newide #10)**

标题： `[C] Implement Council mvp flow`

正文有 **任务 / 目标 / 完成标准 / 不做**，挂 `area: c` 标签。读完就知道做什么、做到什么算完。

✗ **反例 (hci #4、#5)**

两个 Issue 标题正文**完全相同**：“验证了交互逻辑”。无标签、无完成标准。这种既重复又无法追踪。

**规则：** 每个任务一个 Issue；标题 `[工作流] 动词 + 对象`；正文用任务模板；assign 到人；挂标签；放进看板。

### 7.2 统一标签体系（跨仓一致）

类别	标签
模块（可选）	按模块/区域给标签，如 <code>area-driver</code> 、 <code>area-ui</code> （全小写、半角、连字符；别用全角冒号 <code>area: c</code> ）
类型	<code>feature</code> <code>bug</code> <code>contract</code> <code>telemetry</code> <code>docs</code>
优先级	<code>P0</code> <code>P1</code> <code>P2</code>
协作	<code>integration</code> <code>blocked</code> <code>needs-review</code>

**小坑提醒：** newide 的 `area: c` 用了全角冒号 “: ”，跟英文 `:` 不是一个字符，会导致筛选/脚本对不上。统一用**半角、全小写、连字符**。

### 7.3 看板 + 里程碑

- 看板：** 建一个 **跨四仓的 GitHub Project** (org 级)，列：`Backlog → Ready → In Progress → In Review → Done`。Issue/PR 自动流转。WS3 的“看板 mock”是产品功能；这个是团队管理用的项目看板，两回事。
- 里程碑 = 周节点：** 按每周建 milestone（如 `里程碑①`、`里程碑②`、`里程碑③`），每个 Issue 归到对应周，一眼看出本周还剩多少没做完。



Issue 看板：工单随阶段在列间流转。注意——不要手动拖到 Closed，而是由合并 PR 自动关闭对应 Issue。

## 8 仓库治理：搬进一个 GitHub Org

三仓挂在三个个人号下，是最大的治理隐患。本周建一个 Org（如 `ade-team` / `aixcoder-ade`），把三仓 transfer 进去。

- **Teams**：按需建 team、给成员合适的访问权限，权限清晰即可。
- **分支保护 (Ruleset)** 对每个仓的 `main` 开启：
  - ☒ 禁止直推，必须经 PR
  - ☒ 合并前需  $\geq 1$  个 approve
  - ☒ 合并前 CI `verify` 必须通过 (required status check)
  - ☒ 要求分支与 main 保持最新
  - ☒ 合并后自动删分支
- **统一设置**：默认合并方式设为 Squash；关掉 merge commit，保持历史线性。

**transfer 不丢东西**：GitHub transfer 会保留 issue / PR / star / 历史，旧地址自动重定向。各仓主人在 Settings → Transfer ownership 即可。



## 9 本周落地清单：每个仓库要补的 8 件事 ★

定义“完成”：下面 8 件做完，该仓库就达到了协同开发的工程基线。建议优先完成 1-6，再补 7-8。

- ❑ ① 进 Org：把仓库 transfer 到统一 Org，开启 main 分支保护（禁直推 + 需 review + 需 CI）。
- ❑ ② 加 CI：放入 `.github/workflows/ci.yml`，让 `pnpm verify` 在每个 PR 上自动跑。
- ❑ ③ 统一脚手架：补齐 `.editorconfig`、`.nvmrc`、ESLint、Prettier、strict tsconfig，统一 `package.json` scripts。
- ❑ ④ 清垃圾：从仓库（含历史）移除 `gh.zip` / `my-session.json` / `*.tsbuildinfo`，升级 `.gitignore`。
- ❑ ⑤ 装门禁：Husky + lint-staged + commitlint，提交即自动格式化、提交信息强制规范。
- ❑ ⑥ 补治理文件：PR 模板、Issue 模板、CODEOWNERS、CONTRIBUTING、LICENSE。
- ❑ ⑦ 改协作习惯：本周起所有改动走 PR；配好 git 身份（不再有 `unknown`）；提交用 Conventional Commits。
- ❑ ⑧ 对齐契约与看板：把本仓相关契约登记进 `ade-contracts`；Issue 全部挂统一标签、归入当周里程碑、进跨仓看板。

### 各仓的“额外重点”

仓库	除 8 件外，本周尤其要做
acp-client-prototype	把删掉的 SPEC 找回来并冻结；移除 <code>my-session.json</code> ；补 <code>test</code> 脚本让 <code>tests/</code> 能跑
newide-scaffold	把 <code>src/*/contract.ts</code> 抽成契约种子；标签全角冒号改半角；补 CI（工程化最好，差临门一脚）
hci-ide-mvp	删 <code>gh.zip</code> 和 <code>tsbuildinfo</code> ；配 git 身份去掉 <code>unknown</code> ；去重 Issue(#4/#5)、加标签；合并或删除 <code>feature-update-zyc</code> 分支；补 ESLint/Prettier
埋点（跨仓）	先产出必埋点清单（没有它无法接入）；明确 owner；与各仓约定事件埋点位置

## 10 一键脚手架与历史清理

### 一键引导脚本（在每个仓库根目录跑一次）

bootstrap.sh（按需裁剪）

```
1. 锁工具链
echo "20" > .nvmrc

2. 装工程化依赖
pnpm add -D typescript typescript-eslint @eslint/js eslint \
 prettier eslint-config-prettier vitest \
 husky lint-staged @commitlint/cli @commitlint/config-conventional

3. 起预提交门禁
pnpm exec husky init
echo 'pnpm exec lint-staged' > .husky/pre-commit
echo 'pnpm exec commitlint --edit "$1"' > .husky/commit-msg

4. 停止追踪已入库的垃圾
git rm -r --cached --ignore-unmatch *.zip my-session.json *.tsbuildinfo

5. 配好自己的 git 身份（解决 unknown 作者）
git config user.name "你的名字"
git config user.email "你的@邮箱"
```

**彻底清除大文件历史**（如 5.8MB 的 gh.zip）：装 `git-filter-repo` 后跑 `git filter-repo --invert-paths --path gh.zip`，再强推。务必先在 Org 里协调好，因为会改写历史。

### 附录 · 把基础打牢（按需查阅）

## 附录 A · Git 基础：版本库、三区与提交

### 为什么要做版本控制

当你把同一份代码反复保存成 `项目-改`、`项目-最终版`、`项目-最终版2` 时，灾难就已经埋下了：你说不清两份文件到底差在哪里，更不知道某段逻辑是谁、在什么时候、为什么改的；一旦想退回上周那个能跑的状态，往往已经找不回来。多人协作只会让混乱成倍放大。

版本控制系统正是为此而生。它把项目每一次有意义的修改都完整记录下来，让你随时能看清“过去发生了什么”、把代码回退到任意历史状态，并在两个人改了同一处时把改动安全地合到一起。Git 是目前业界事实上的标准，它是分布式的——你电脑上有一份完整的历史，绝大多数操作（查看历史、提交、比较差异）都在本地瞬间完成，不依赖网络。

### 把项目纳入版本控制：git init 与 git clone

开始用 Git 管理一个目录，只需在该目录下初始化一个本地版本库。Git 会创建一个隐藏的 `.git` 文件夹，所有历史和元数据都存在那里——它就是“版本库”本身，平时不需要去动它。

从零开始：在当前目录初始化版本库

```
git init
```

如果项目已经托管在 GitHub 上，你更常用的是把它“克隆”到本地。克隆会一次性把远端的全部文件和完整历史拉到本地，并自动建立与远端的关联。

克隆一个已有的远端仓库

```
git clone https://github.com/<用户名>/<仓库名>.git
```

**第一件事：先配好身份。**初次在一台机器上使用 Git，务必先设置提交身份。否则 Git 不知道该如何为你的提交署名，作者信息可能显示为 `unknown`，弄脏整段提交历史——这是真实踩过的坑，事后清理代价很大。建议把邮箱设成你 GitHub 账号绑定的邮箱，提交才能正确归属到你名下。

一次性配置（全局生效，只需做一次）

```
git config --global user.name "你的名字"
git config --global user.email "你@example.com"
```

### 三区模型：工作区、暂存区、版本库

理解 Git 的关键，是看懂文件在三个区域之间的流动：

- **工作区：**你实际编辑文件的目录，也就是你能直接看到、打开的那些文件夹和文件。
- **暂存区：**一块“待提交清单”。你挑选哪些改动准备进入下一次提交，就把它们放进这里。
- **版本库：**`.git` 目录中真正保存历史快照的地方。一旦提交，这份快照就被永久记录下来。

这三者分工明确：你在工作区改动，用 `git add` 把想要的改动挑进暂存区，再用 `git commit` 把暂存区的内容定格成一次提交。多出来的暂存这一步，正是 Git 灵活的地方——它让你能精确控制“这一次提交到底包含哪些改动”，而不是把所有零散修改一股脑塞进去。

#### git add：把改动放进暂存区

无论是新建的文件，还是已被跟踪文件的修改，都需要先 `git add` 才会被纳入下一次提交。它的作用就是把工作区里挑中的改动复制一份到暂存区。

将改动加入暂存区

```
git add 文件名 # 暂存指定文件
git add . # 暂存当前目录下的所有改动
```



git add 把工作区里挑中的改动放进暂存区。

**git commit: 把暂存区定格为一次提交**

当暂存区里就是你想要的一组改动时，用 `git commit` 把它们打包成一次提交，连同一条说明信息一起写入版本库。这条信息是写给未来的自己和队友看的，请写清楚“这次改了什么、为什么改”。

提交暂存区的内容

```
git commit -m "feat: 添加用户登录页面"
```



改动从工作区到暂存区，再由 `git commit` 写入版本库，成为一次永久快照。

**git status 与 git log: 随时看清局面**

这两条是你最常用的“只看不改”命令。`git status` 告诉你当前哪些文件已暂存、哪些被修改了还没暂存、哪些尚未被跟踪；`git log` 则列出历史提交，是回溯“过去发生了什么”的入口。

查看状态与历史

```
git status # 查看工作区与暂存区的当前状态
git log # 查看提交历史
git log --oneline --graph # 一行一条、带分支图，更紧凑
```

**撤销也用现代命令。** 改乱了想丢弃工作区的修改，用 `git restore 文件名`；只想把文件移出暂存区、但保留改动，用 `git restore --staged 文件名`。切换或新建分支则用 `git switch 分支名`（新建加 `-c`）。这些比老式的 `git checkout` 语义更清晰，不易误操作。

## .gitignore：哪些东西绝不入库

不是目录里所有文件都被 Git 跟踪。在仓库根目录放一个 `.gitignore` 文件，列出需要忽略的路径，Git 就会自动跳过它们。三类东西尤其不该入库：

- **构建产物与依赖目录**：如 `node_modules/`、`dist/`、编译输出。它们可以由源码和依赖清单重新生成，入库只会让仓库臃肿、合并时频繁冲突。
- **密钥与凭据**：如 `.env`、密码、API Token、私钥。一旦提交进历史，即使后来删除也会留在记录里，等同于泄露。
- **大文件与本地杂物**：如视频、数据集、编辑器和系统生成的临时文件（`.DS_Store` 等）。

.gitignore 示例

```
依赖与构建产物
node_modules/
dist/

密钥与本地配置
.env

系统与编辑器杂物
.DS_Store
```

本手册采用 `pnpm` 管理依赖，因此只需提交 `package.json` 与 `pnpm-lock.yaml`，由它们锁定版本；`node_modules/` 永远交给 `pnpm install` 在本地还原，绝不入库。

**最佳实践。** 小步提交、勤提交——每个提交只做一件能说清楚的事，提交信息遵循 Conventional Commits 规范（如 `feat`：新功能、`fix`：修缺陷、`docs`：改文档），历史才读得懂、回退才精准。开工前先 `git status` 看清局面，提交前再 `git status` 确认这次到底带进了哪些改动；新克隆的仓库第一时间检查并完善 `.gitignore`，从源头上杜绝密钥和构建产物混进版本库。

## 附录 B · 分支与切换

### 分支是什么：一个会移动的指针

很多人以为新建分支会把整个项目复制一份，其实不会。在 Git 里，每次提交都会形成一条首尾相连的版本链；而**分支只是一个指向某次提交的指针**。新建分支，不过是再造一个指针指向当前这次提交，所以无论建分支还是切分支，代价都极小，几乎是瞬间完成。

正因为分支是指针，不同分支“看到”的版本链也不同：从一个分支回溯到它指向的提交、再沿着父提交一路往前，得到的就是这条分支视角下的历史。



分支只是指针，所以每条分支看到的版本链各有不同。

### HEAD：你现在站在哪里

还有一个关键指针叫 **HEAD**，它表示**当前正呈现在工作区里的版本**，通常指向你所在的那条分支。切换分支，本质就是把 **HEAD** 指过去，工作区随之变成那条分支的内容；之后你产生的新提交，也会顺着 **HEAD** 所在的分支继续延伸。

### 怎么做：新建、切换与查看

推荐使用现代写法 `git switch` 和 `git restore`，它们语义清晰、不易误用（老命令 `git checkout` 一个词承担了太多职责）。

#### 分支的日常操作

```
查看本地所有分支，带 * 的是当前分支
git branch

新建一个分支并立刻切换过去（开发新功能时常用）
git switch -c feature/login

在已有分支之间来回切换
git switch main
git switch feature/login

把工作区某个文件恢复到上次提交的样子
git restore src/app.ts
```

**提示：**分支名建议带上语义前缀，例如 `feature/`、`fix/`，再配合简短描述，让别人一眼看懂这条分支在做什么。

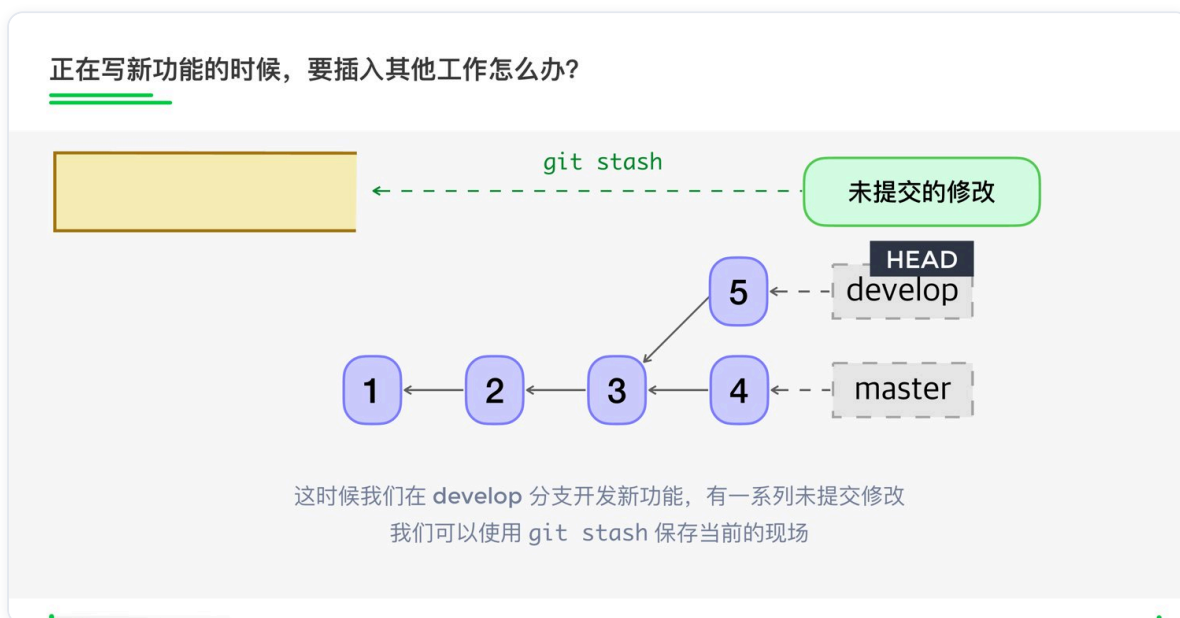
## 短命分支：一条分支只干一件事

分支要“短命”才好管理。一个实用的约定是：**一条分支 ≈ 一个 Issue ≈ 1 到 3 天的工作量**。范围越小，提交越聚焦，对应的 Pull Request 也越好评审、越快合并；拖得越久，分支越容易和主干分叉、积累冲突。

**约定：**每条功能分支都从最新的 `main` 切出，对应一个明确的 Issue；功能做完就尽快开 Pull Request 合并，不要让分支长期挂着。

## 写一半被插队：用 git stash 收起现场

开发途中常会被临时插进来的活打断——这时你手上有一堆改了一半、还不适合提交的代码。直接切分支会把这些改动带过去，造成混乱。`git stash` 正是为此而生：它把工作区里未提交的修改整体“收起来”暂存，让工作区回到干净状态，你就能安心切去处理别的事；忙完再把现场原样取回来。



`git stash` 把写一半的现场临时收起，腾出手处理插队的任务。

临时收起现场再恢复

```
把当前未提交的改动收进暂存堆
git stash

切去处理插进来的活
git switch fix/urgent-bug
.....处理完，提交.....

切回原分支，把之前的现场取回来
git switch feature/login
git stash pop
```

## 合并之后：及时删掉分支

当 Pull Request 以 Squash & merge 合入主干后，这条功能分支的使命就结束了。它已经合进 `main`，再留着只会让分支列表越来越乱。养成习惯：合并后顺手删掉本地和远端的分支。

合并后清理

```
删除已合并的本地分支
git branch -d feature/login

删除对应的远端分支
git push origin --delete feature/login
```

**最佳实践：**从最新主干切出一条短命分支 → 用 `git switch -c` 开工 → 被插队就 `git stash` 收起现场 → 功能完成开 Pull Request 走 Squash & merge → 合并后立即删除分支。把这套流程跑顺，你的提交历史会始终清爽、可追溯。



## 附录 C · 合并、变基与冲突

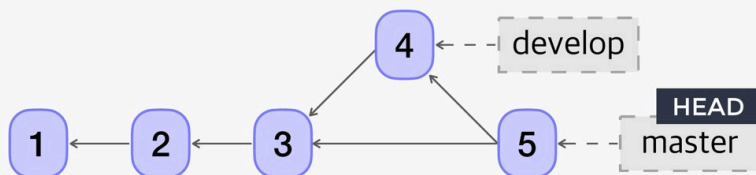
当多个人在不同分支上推进同一个项目，最终都要把改动汇合到一起。Git 提供了两条主要路径：合并（merge）把两条历史拼接起来，变基（rebase）把一段提交搬到新的起点上重放。理解它们的差异，是写出清晰提交历史、并在冲突来临时从容应对的基础。

### C.1 两种合并：fast-forward 与三方合并

合并的结果取决于两条分支的相对位置。如果目标分支自分叉以来没有任何新提交，待合并分支只是单纯地“走在前面”，那么 Git 只需把目标分支的指针向前挪到最新提交即可——这就是 **fast-forward（快进）**。它不产生任何新提交，历史依旧是一条直线。

但更常见的情况是：两条分支各自都有了新提交，谁也不是谁的直接祖先。此时无法快进，Git 会找到两者共同的祖先，做一次**三方合并（non-fast-forward）**，并自动生成一个全新的**合并提交**。这个提交有两个父节点，分别指向被合并的两条分支，历史从此出现一个“菱形”的分叉与汇合。

将 **develop** 分支合并到 **master** 分支



non-fast-forward 合并会产生一个新的提交

当两条分支各有新提交时，三方合并会创建一个带两个父节点的合并提交。

**提示：**合并方向很重要。要把 **feature** 并入主干，应先切到目标分支再合并：先 `git switch main`，再 `git merge feature`。被合并的是“当前分支以外”的那条。

### C.2 冲突长什么样，怎么读、怎么解

三方合并并不总能自动完成。只要两条分支改动的是文件的**不同部分**，Git 就足够聪明，会把两边的改动各取所需拼到一起，无需人工介入。下图就是这种理想情况：一边新增了片段、一边修改了另一段，互不打架。



两条分支改的是同一文件的不同区域，Git 能自动合并，无需手动处理。

真正需要你出手的，是两条分支改到了同一处且内容不一致——Git 无法替你判断该听谁的，于是把抉择权交给你，在文件里写入冲突标记：

conflicted.ts（出现冲突时的样子）

```
function greet(name) {
<<<<<<< HEAD
 return `你好, ${name}`; // 当前分支的版本
=====
 return `Hi, ${name}`; // 待合并分支的版本
>>>>>>> feature/i18n
}
```

读法很直接：<<<<<<< 到 ===== 之间是当前分支（HEAD）的内容，===== 到 >>>>>>> 之间是待合并分支的内容。解决冲突，就是把这段编辑成你最终想要的样子——可以二选一，也可以融合两者——同时删掉全部三行标记。改完后把文件重新加入暂存区，再提交，合并即告完成：

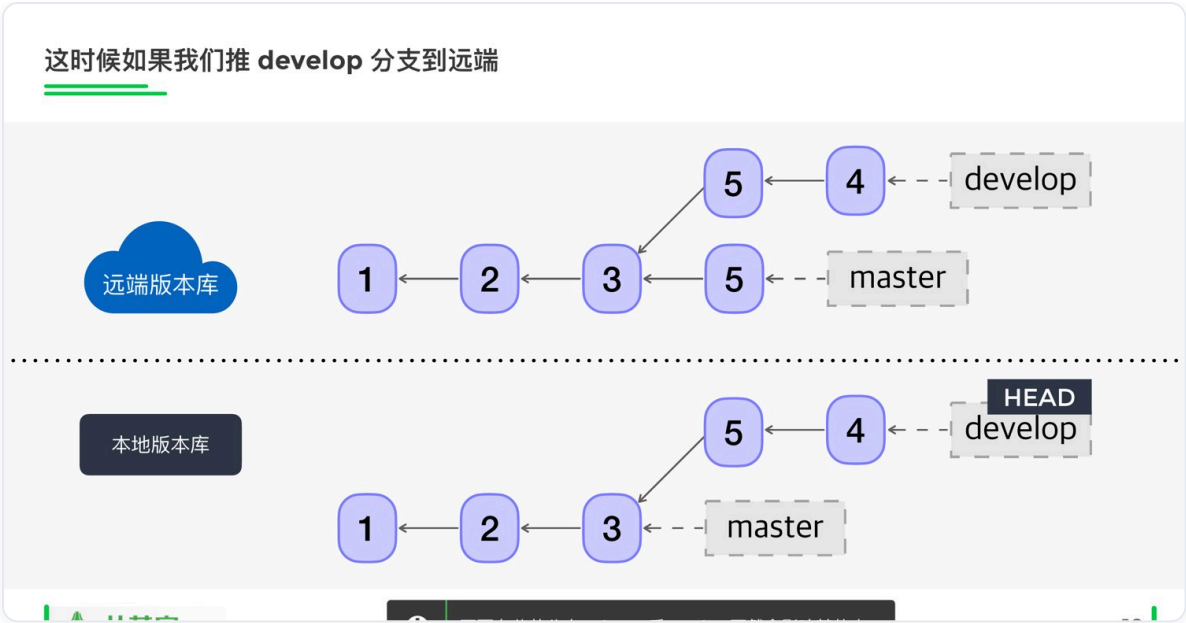
解决冲突后

```
手动编辑文件，留下正确内容并删除冲突标记
git add conflicted.ts
git commit # 完成这次合并提交
```

**坑：**解决冲突时千万别漏删 <<<<<<<、=====、>>>>>>> 这些标记，它们会被原样留进代码导致编译或运行报错。改之前若想反悔，`git merge --abort` 可一键回到合并前的状态。

### C.3 用 rebase 整理出线性历史

反复合并会让历史长出许多分叉与合并提交，回看时像一团乱麻。**rebase（变基）**提供了另一种思路：它把你这条分支上的提交逐个“摘下来”，再依次重放到目标分支的最新提交之上，相当于假装你从一开始就是基于最新代码开发的。结果是一条干净的**线性历史**，没有多余的合并提交。



变基把本地提交搬到目标分支顶端重放，历史保持一条直线。

常见用法是在更新分支时顺手变基，让自己的提交始终长在最新主干之上：

```
用变基更新分支

git switch feature
git fetch origin
git rebase origin/main # 把本地提交重放到最新 main 之上
拉取远端更新时也可直接：
git pull --rebase origin main
```

变基过程中同样可能遇到冲突，处理方式与合并一致：编辑文件、删除标记、`git add`，然后用 `git rebase --continue` 接着重放下一个提交，逐个解决直到完成。中途想放弃，用 `git rebase --abort` 退回原点。

**黄金规则：**绝不要对已经推送到公共分支的提交做 rebase 后强推。变基会重写提交、改变它们的身份，一旦 `git push --force` 覆盖远端，所有基于旧历史工作的协作者都会陷入混乱。**只对尚未分享的本地提交变基**，公共历史一律用合并来推进。

C.4 我们的默认约定

场景	做法
合并 Pull Request	统一使用 <b>Squash &amp; merge</b> ，把一个 PR 的多次提交压成一个，保持主干历史简洁可读
提交信息	遵循 Conventional Commits，如 <b>feat:</b> 新增登录页、 <b>fix:</b> 修复空指针
PR 合并前落后于主干	本地 rebase 到最新主干后再更新分支，确保改动基于最新代码、CI 真实可信
依赖管理	统一使用 pnpm，提交锁文件 <code>pnpm-lock.yaml</code>

**最佳实践：**把规则记成一句话——**对内随意，对外谨慎**。本地未推送的提交尽管用 rebase 梳理整齐；面向团队的 PR 一律走 Squash & merge 落入主干，需要同步主干时用 `git pull --rebase` 或本地变基更新分支，但永远不对已共享的历史强推。这样既有线性清爽的主干，又不会踩坏任何人的本地仓库。

## 附录 D · 远端协作与 Pull Request

本地仓库让你独自记录历史，而真正的团队开发需要一个所有人都能访问的“中转站”——这就是**远端仓库**。远端仓库托管在 GitHub 上，每个人把自己本地的提交推上去、再把别人的提交拉下来，从而保持协作同步。理解清楚“本地”与“远端”之间的几条数据通道，是顺畅协作的前提。

### 远端与 origin

当你用 `git clone` 把一个 GitHub 仓库下载到本地时，Git 会自动为这个远端起一个默认名字 `origin`。之后所有“拉取”“推送”默认都针对它。你可以随时查看当前关联了哪些远端：

查看远端配置

```
git remote -v
origin git@github.com:team/project.git (fetch)
origin git@github.com:team/project.git (push)
```

如果项目是在本地用 `git init` 新建、还没有任何远端，可以手动关联一个：

手动添加远端

```
git remote add origin git@github.com:team/project.git
```

### 克隆、拉取与推送

三个核心动作分别对应三个方向的数据流动，务必分清。

命令	作用	是否改动你的工作目录
<code>git clone &lt;地址&gt;</code>	首次把整个远端仓库复制到本地	是（生成新目录）
<code>git fetch</code>	把远端的最新提交下载到本地，但 <b>不动</b> 你正在编辑的文件	否
<code>git pull</code>	等于 <code>fetch</code> 再加一次合并，会更新当前分支的文件	是
<code>git push</code>	把本地已提交的内容上传到远端	否（改动的是远端）

**fetch 与 pull 的区别：**`git fetch` 只“取回信息”，让你先看清远端发生了什么、再决定如何合并，更安全；`git pull` 则“取回并立即合并”，省事但合并是自动发生的。日常想稳妥时先 `fetch` 查看差异，确认无误再合并。

### 用 -u 建立分支跟踪

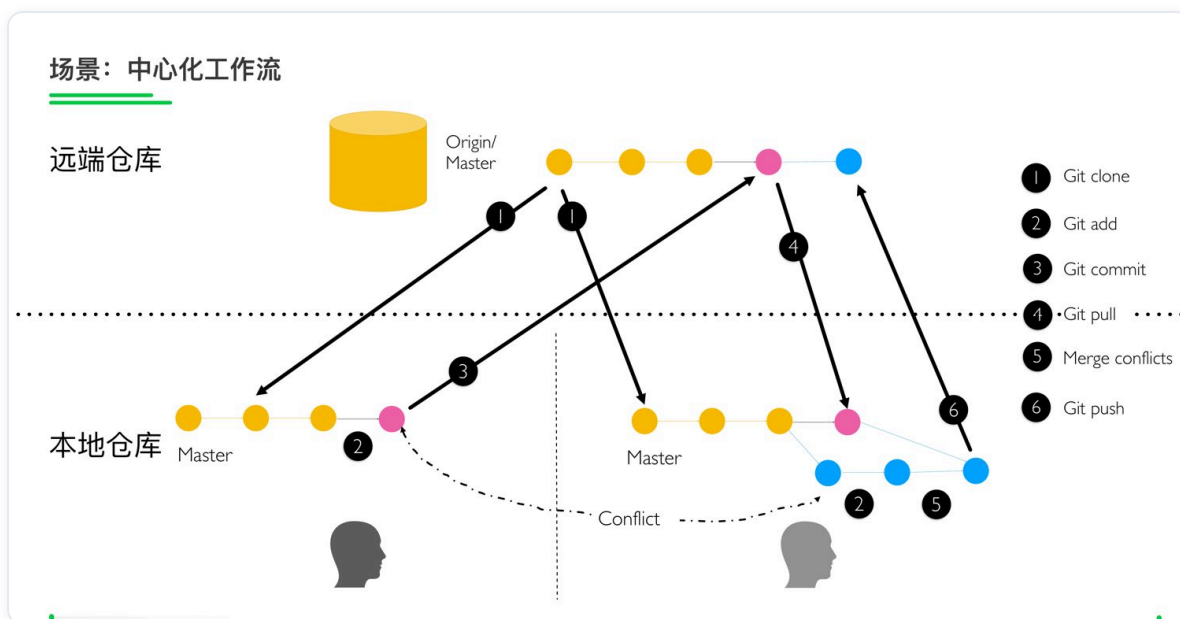
第一次把一个新分支推到远端时，本地分支还不知道自己对应远端的哪一条分支。用 `-u`（即 `--set-upstream`）建立这种“跟踪关系”，之后再推拉就只需敲 `git push` / `git pull`，无需每次写全：

首次推送并建立跟踪

```
git push -u origin feat/login
之后简化为：
git push
git pull
```

### 多人共享同一远端时的协作

当多个人对着同一个远端 `clone` / `pull` / `push` 时，难免有人和你改动同一处代码。此时推送会被拒绝，提示你先把远端的新提交拉下来；如果两边改了同一行，就会出现**冲突**，需要手动选择保留哪部分后再提交、再推送。下图展示了这条“克隆 → 改动 → 拉取合并 → 解决冲突 → 推送”的完整回路。



多人对同一远端协作：推送前先拉取，遇到冲突先解决再推送。

## 功能分支工作流：通过 PR 进入主干

专业团队几乎从不直接往 `main` 推代码，而是采用**功能分支工作流**：每项改动都先在自己的分支上完成，再通过 GitHub 的 **Pull Request (PR)** 请求把它合入 `main`。PR 是一道“审查闸门”——队友可以在合并前阅读你的改动、留下评论、运行检查。完整流程如下：

1. 从最新的 `main` 切出一条功能分支（命令用现代写法 `git switch -c`）；
2. 在分支上正常编码、提交，提交信息遵循 **Conventional Commits**（如 `feat: 新增登录页`）；
3. 把分支推到远端并建立跟踪；
4. 在 GitHub 上对着这条分支开一个 PR，指向 `main`；
5. 队友评审通过后，用 **Squash & merge** 合并，改动才进入 `main`。

### 功能分支工作流常用命令

```
1. 先同步主干，再切出功能分支
git switch main
git pull
git switch -c feat/login

2. 编码后提交（遵循 Conventional Commits）
git add .
git commit -m "feat: 新增登录页表单校验"

3. 推送并建立跟踪，然后到 GitHub 开 PR
git push -u origin feat/login
```

**约定：**受保护的 `main` 分支禁止直接 `push`，所有改动一律通过 PR 进入。如需丢弃本地未提交的改动，用现代写法 `git restore <文件>` 而非旧的 `checkout`。

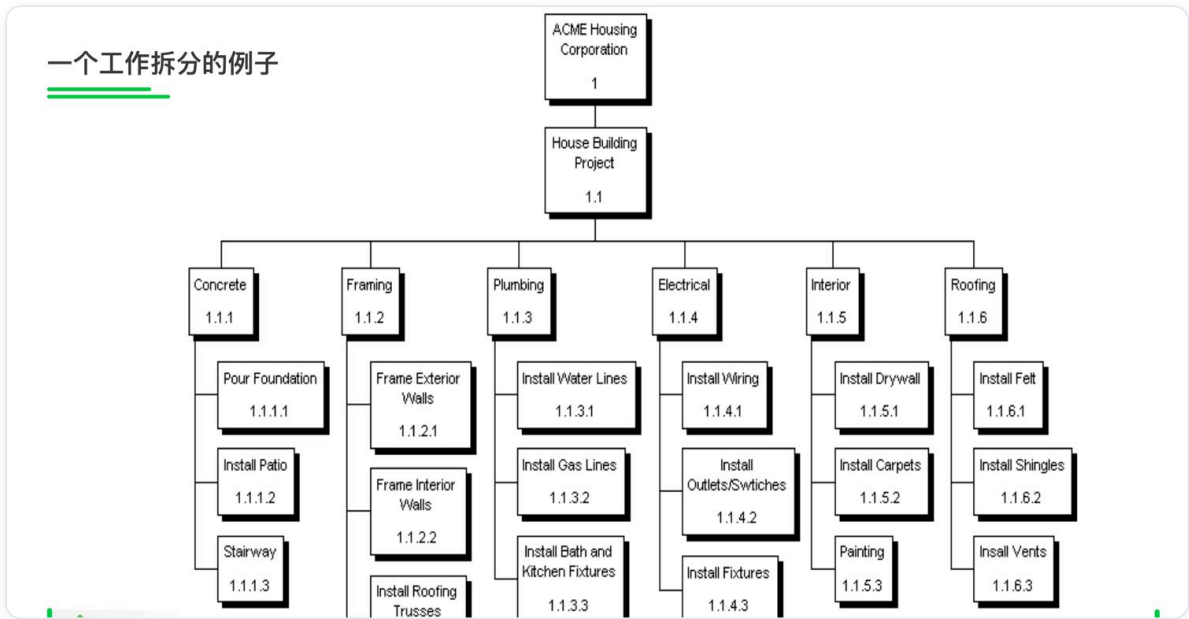
**最佳实践：**动手前先 `git pull` 同步主干，让功能分支从最新代码出发；分支保持“小而专”，一条分支只做一件事，PR 才好评审、合并冲突也更少。用 `git fetch` 勤看远端动态，避免推送时才发现落后太多。包安装统一用 `pnpm install`，确保团队依赖版本一致。

## 附录 E · 用 GitHub 做项目管理

写代码只是项目的一部分。一个项目能不能按时交付，往往取决于动手之前有没有把事情想清楚、拆明白，以及过程中有没有持续追踪进度。这一节讲的是如何把任务拆开、把工期估准，并用 GitHub 自带的 Issue、Projects 和里程碑把整个开发过程管起来。

### 先把大任务拆小：工作分解（WBS）

面对一个大需求，直接开干几乎一定会漏掉东西、也估不准时间。正确的做法是自上而下地把它拆成一层层可管理的小任务：先按产品、过程、组织等维度切成几大块，再把每一块继续往下拆，直到每个叶子任务都小到一个人能在可预期的时间内独立完成。这种树状的拆分结果就叫工作分解结构（Work Breakdown Structure，简称 WBS）。



自上而下逐层拆分，直到每个叶子任务都可独立完成、便于估时。

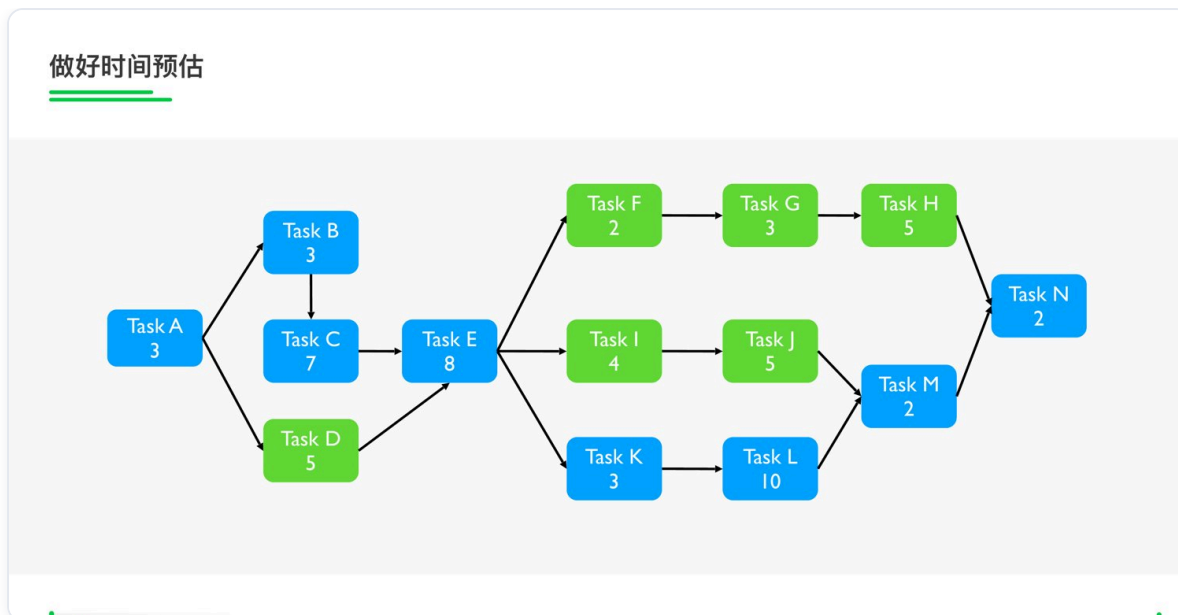
**拆到什么程度：**一个好的叶子任务应当能用一句话说清交付物，且工期通常在一到几天之内。拆解时要兼顾全局，确保覆盖完整、没有遗漏，同时也要现实——别拆出无法落地的任务。

### 看清依赖、估准工期：任务依赖图与关键路径

任务拆完之后，它们并不是各自独立的——有些任务必须等前置任务完成才能开始。把这种先后关系画成一张有向图，每个节点标上自己的预估工期，就能看出整个项目的节奏。

在这张图里，从起点到终点、累计耗时最长的那条路径，就是**关键路径**。它决定了整个项目最快能在多久完成：关键路径上任何一个任务延期，整个项目就会跟着延期；而不在关键路径上的任务，则有一定的缓冲余量。找到关键路径，既能给出靠谱的总工期估计，也能让你一眼看出该重点保障哪些任务、瓶颈在哪里。





沿依赖图累计工期最长的路径即关键路径，用来排期并定位瓶颈。

**延期是怎么积累的：**一个项目很少因为某一次大事故而延误一年，更多是关键路径上每个任务都“只晚一天”，一路累加出来的。所以排期时要给关键路径留出余量，并在过程中持续盯着它。

## 把任务变成 GitHub Issue

拆好的每个小任务，都应该在 GitHub 上对应一个 **Issue (工单)**。Issue 是团队协作的最小追踪单元，一个写得好的 Issue 应当包含：

- **标题：**用一句话说清这个工单要完成什么，简洁可检索。
- **正文：**交代背景、是否已做过前期调研与设计、当前方案是什么；有界面效果图就贴上。Issue 正文支持 Markdown，可以列清单、贴代码、附图。
- **完成标准：**明确写出“做到什么程度算完成”，这样后续才能客观判断它能否被关闭。Issue 都应当是可以被明确完成、关闭的。
- **标签 (Labels)：**建立时就打上类别与优先级标签，方便筛选和排序。
- **指派 (Assignee)：**在第一行代码提交之前，这个 Issue 就应该已经有了负责人。

**善用 AI 起草：**可以让 AI 工具帮你对一个 Issue 的背景和拆解写出初稿，你只做确认与补充，把精力留给真正需要判断的部分。

## 提交里用 Closes #N 关联并自动关闭 Issue

开发某个任务前，先从主分支切出一条对应该 Issue 的开发分支，在上面完成工作。每次提交时，都在提交信息里写明它关联到哪个 Issue：用 `Closes #N` 关联，当这次提交（或它所在的 Pull Request）合入默认分支时，编号为 N 的 Issue 会被自动关闭；如果只是关联但还不打算关闭，用 `Ref #N`。

提交信息中关联 Issue

```
git commit -m "refactor: fix indent style. Closes #12"
git commit -m "feat: add support to course switch. Ref #182"
```



**井号前要留空格：** # 紧贴在前一个词后面（如 `style.Closes#12`）不会被识别，必须写成 `. Closes #12`，即 # 前有  
空格、# 与数字之间无空格。提交信息建议遵循 Conventional Commits 规范，用 `feat`、`fix`、`refactor` 等前缀开  
头。

开发流程上推荐用现代命令：用 `git switch -c` 新建并切换分支，用 `git restore` 撤销工作区改动。完成后发起 Pull  
Request (PR)，在 PR 描述里同样写上 `Closes #N`，并指定 Reviewer。我们默认采用 **Squash & merge** 合并，把  
开发分支上的零碎提交压成一条，保持主分支提交线清晰；合并后顺手删除已无用的源分支。

新建分支并撤销改动（现代写法）

```
git switch -c feat/course-switch # 从当前分支创建并切换
git restore src/app.ts # 丢弃 src/app.ts 的未暂存改动
git restore --staged src/app.ts # 仅取消暂存，保留改动
```

**关闭 Issue 的方式：**不要手动把工单拖到 Done 或点关闭，而应让对应 PR 被合并时通过 `Closes #N` 自动关闭。这样工单  
的关闭总是与真实的代码合入绑定，记录可追溯。

用 GitHub Projects 看板追踪进度

把一个团队（或仓库）的所有 Issue 汇总到一块 **GitHub Projects** 看板上，每个工单显示为一张卡片，按所处阶段分列  
摆放。常用的列从左到右是：

列	含义
Backlog	已建立但尚未开始的待办工单
In Progress	正在开发中的工单
In Review	开发完成、已发起 PR 等待复审与测试
Done	对应 PR 已合并、工单自动关闭

开发过程中，随着任务推进，及时把卡片从左往右拖动：开工时拖到 In Progress，发起 PR 时拖到 In Review。这样整  
张看板就实时反映了项目进度，谁卡住了、哪一列堆积了，一目了然。

用里程碑标出周节点

**里程碑 (Milestone)** 用来把一批 Issue 归集到一个固定的时间段里，通常对应每周的节点，比如“第三周：完成开发  
与联调”。给里程碑设定起止日期，再把相关 Issue 关联进去，GitHub 就会自动统计这一阶段的完成进度（已关闭 / 总  
数），让你清楚地知道本周该交付的东西完成了多少、是否临近截止。建立 Issue 时就顺手关联好里程碑，排期会清晰  
很多。

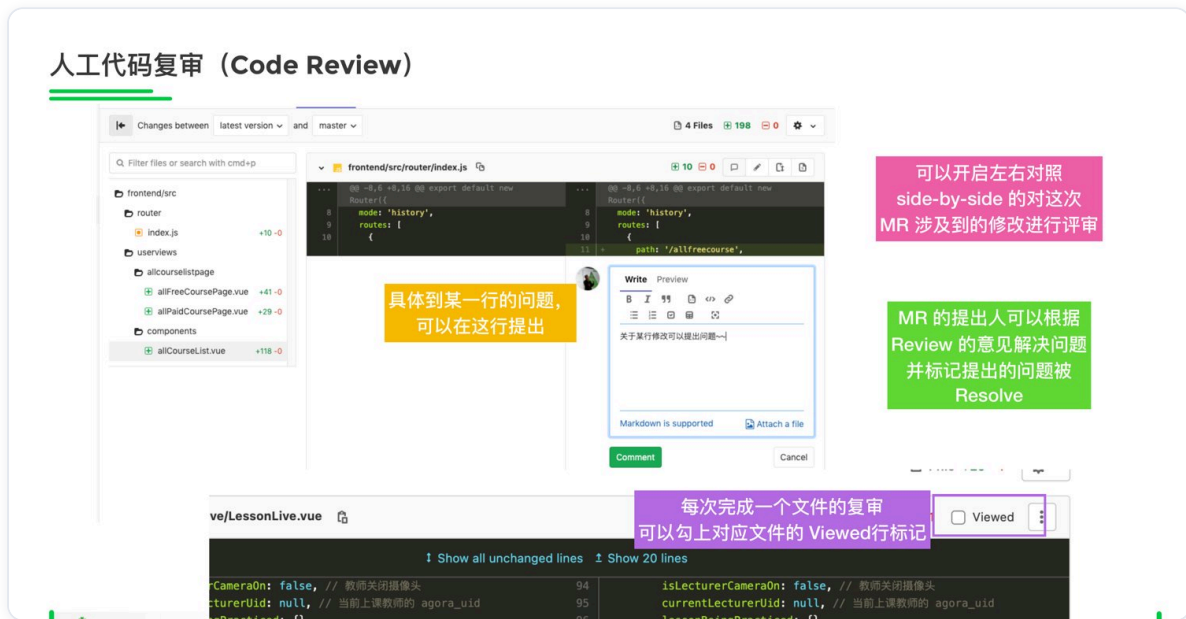
**本节小结：**动手前先用 WBS 把大任务拆小，借助依赖图与关键路径估准工期；每个任务建成一个标题清晰、有完成标准、  
打好标签并指派到人的 Issue；提交和 PR 里用 `Closes #N`（记得 # 前留空格）关联并自动关闭工单，按 Squash &  
merge 合入主分支；最后用 Projects 看板（Backlog → In Progress → In Review → Done）追踪进度，用每周里程碑把  
握节奏。把流程跑顺，进度就始终可见、可控。

## 附录 F · 代码评审与 CI/CD

当你在一个分支上写完一段功能，把它推到 GitHub 并开一个 **Pull Request (PR)** 之后，代码并不会立刻进入主干。它要先经过两道关：一道是机器自动跑的检查（CI），一道是真人逐行看的评审（Code Review）。这两道关的目的只有一个——在问题进入 **main** 之前就把它拦住。问题发现得越早，修复成本越低；而且评审还顺带把知识在团队里传开了，让别人也读得懂、改得动你的代码。

### 评审时到底在看什么

一次好的评审不是把代码跑起来、没报错就点通过。评审者要把这段改动当成自己刚写完的代码那样去读，读到能快速理解每一处为什么这么写。常见的关注点包括：拼写与命名是否清晰、有没有不必要的复杂写法、是否有被反复实现、可以抽出来的逻辑、关键路径是否缺了单元测试，以及实现有没有偏离需求或设计。



评审界面：左右对照看 diff，针对某一行直接留评论，看完一个文件就勾上 Viewed。

在 PR 的 **Files changed** 页里，可以开启左右对照（side-by-side）来看每个文件改了什么；针对具体某一行的问题，直接点那一行留下行内评论，讨论就钉在代码旁边，而不是飘在群聊里。每看完一个文件，就勾上它右上角的 **Viewed** 标记——大型 PR 涉及十几个文件时，这能让你清楚自己还剩哪些没看，不至于漏掉。

**规则：**留评论时请区分两类意见。**必须改 (blocking)** 是合并前一定要解决的问题，比如逻辑错误、缺测试、偏离需求；**建议 (nit)** 是无伤大雅的小优化，留给作者自行取舍。明确标出哪条是 blocking、哪条是 nit，作者才知道该先处理什么。

### 返工怎么闭环

评审提出意见后，作者不需要新开 PR，也不要改别的分支。直接在**同一个分支**上按意见修改，提交（遵循 Conventional Commits 规范，比如 **fix: 修正空指针判断**），再推到远端的这个分支——PR 会自动带上新的提交，评审者刷新就能看到更新后的 diff 与新一轮讨论。每条意见被处理后标记为 **Resolved**，直到所有讨论都关闭、参与评审的人都认为没问题了，PR 才算评审通过。

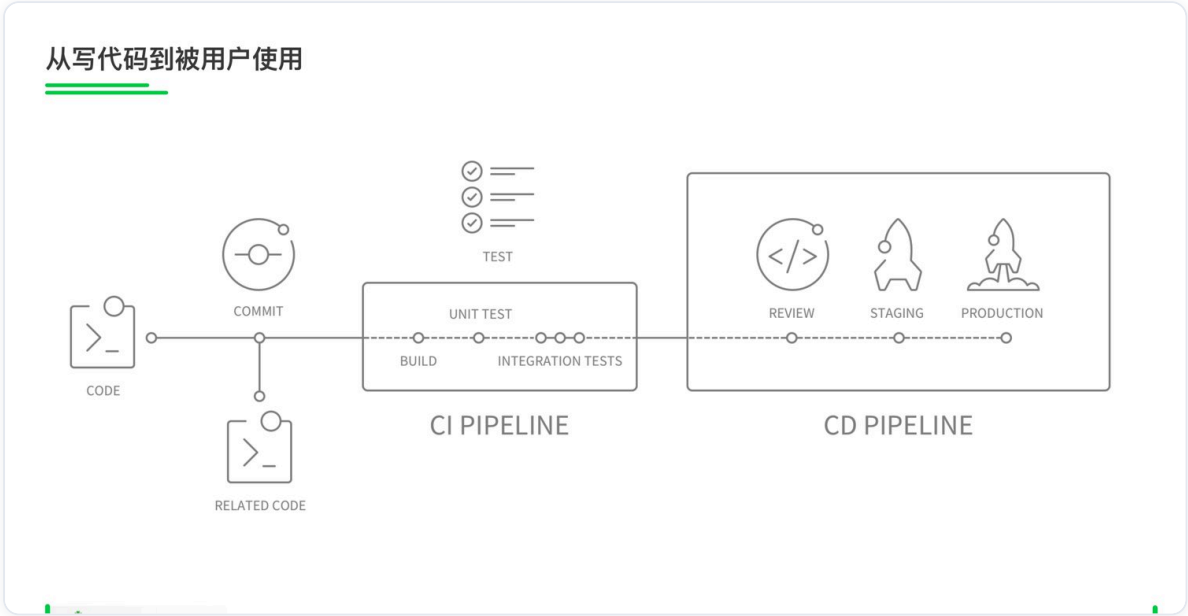
在同一分支上完成返工

```
git switch feature/653-free-course-list # 仍在原分支上
按评审意见修改文件.....
git add -p # 只暂存与本次意见相关的改动
```

```
git commit -m "fix: 移除内联样式并补全单元测试"
git push # 推到远端同名分支，PR 自动更新
```

CI：每个 PR 自动跑的检查

从写代码到被用户用上，中间是一条流水线：CI（持续集成）负责构建、单元测试、集成测试；CD（持续交付/部署）负责把通过的代码送往评审、预发、最后到生产环境。



CODE → CI（构建 · 单元测试 · 集成测试）→ CD（评审 · 预发 · 生产）的完整流水线。

每当你开一个 PR 或往它对应的分支再推提交，CI 都会在那次改动上自动跑一遍 `pnpm verify`（一般串起依赖安装、类型检查、Lint、单元测试与构建）。你不必手动触发，绿勾代表这次改动通过了机器这一关，红叉则会直接告诉你哪一步挂了。

本地先跑一遍，别把红叉推上去

```
pnpm install # 安装依赖
pnpm verify # 本地跑与 CI 相同的检查：类型 / lint / 单测 / 构建
全绿之后再 git push，能省下一来回等流水线的时间
```

合并门禁与合并后的复查

能不能合并，由两个硬条件共同决定，缺一不可：

门禁条件	含义
CI 全绿	这次改动上的 <code>pnpm verify</code> 流水线全部通过
≥ 1 个 approve	至少一位评审者点了 Approve，且所有评论已 Resolved

两个条件都满足后，由 PR 发起人或仓库负责人完成合并。默认采用 **Squash & merge**：把这个 PR 上的多次提交压成一条干净的提交进入 `main`，提交信息里带上关联的 issue 编号（如 `Closes #653`），合并后顺手删掉源分支。这样主干历史一个 PR 一条记录，清爽又可追溯。

合并并不是终点。代码进了 `main` 之后，CI 会在 **main 上再跑一次完整验证**。因为 PR 通过时是基于当时的分支，而合并那一刻主干可能已经前进，只有 main 上这次再验证绿了，才能确认你的改动和别人新合入的代码放在一起仍然没问题。

**最佳实践：**推之前先在本地 `pnpm verify` 跑绿，别让 CI 替你发现低级错误；PR 拆小、单一目的，让评审者读得动；留评论对事不对人，分清 blocking 与 nit；返工就在原分支上改了再推，让讨论闭环在同一个 PR 里。把这套流程走顺，合并进 `main` 的每一行代码都是被机器和人一起把过关的。

**一句话收尾：**这套基线，是把“个人能写代码”升级成“团队能交付软件”的必经一步。基线立起来，联调时才能把精力花在“功能拼不拼得通”，而不是“环境为什么又挂了”。

工程协作基线 v1 · 2026-06